NAME : M.GURU BHAGAVAN

REG NO : 192312501

Candidate-Elimination Algorithm

Finding the Version Space from Training Data in a CSV File

Five Problems with Aim, Algorithm, Program, Input and Output

Introduction

The Candidate-Elimination algorithm is a concept-learning algorithm that outputs a description of all hypotheses consistent with the training examples, known as the version space. Instead of tracking a single hypothesis as FIND-S does, it keeps two boundaries: the specific boundary S and the general boundary G. Positive examples generalise S and prune G; negative examples specialise G and prune S. Every hypothesis between S and G is consistent with the data.

Hypothesis notation: each attribute is a specific value (e.g. Sunny), "?" meaning any value is acceptable, or "0" meaning no value is acceptable.

CSV format: the first row holds the column names, every following row is one example, and the last column is the target label (Yes = positive, No = negative).

Limitations: the algorithm assumes noise-free data and that the target concept is a conjunction of attribute constraints. If the data is inconsistent, the version space collapses to empty, and the program reports this.

General Candidate-Elimination Algorithm

1. Initialise G to the set of maximally general hypotheses in H: G = { <?, ?, ..., ?> }.
2. Initialise S to the set of maximally specific hypotheses in H: S = { <0, 0, ..., 0> }.
3. For each training example d:
    a. If d is a positive example:
        i. Remove from G any hypothesis that is inconsistent with d.
        ii. For each hypothesis s in S that is inconsistent with d: remove s from S, add all minimal generalisations h of s such that h is consistent with d and some member of G is more general than h, then remove from S any hypothesis more general than another in S.
    b. If d is a negative example:
        i. Remove from S any hypothesis that is inconsistent with d.

ii. For each hypothesis g in G that is inconsistent with d: remove g from G, add all minimal specialisations h of g such that h is consistent with d and some member of S is more specific than h, then remove from G any hypothesis less general than another in G.

4. Output the boundary sets S and G; the version space is every hypothesis lying between them.

Problem 1: EnjoySport Prediction

Aim

To implement the Candidate-Elimination algorithm in Python, reading training examples from a CSV file, and to output the set of all hypotheses consistent with the data for the EnjoySport concept.

Problem Statement

Given weather observations (Sky, AirTemp, Humidity, Wind, Water, Forecast) stored in a CSV file, with the label EnjoySport (Yes/No), find the specific boundary S, the general boundary G and the complete version space.

Algorithm

Step 1: Read the CSV file; store the attribute names, the examples X and the labels y (Yes = positive).

Step 2: Initialise S = <0, 0, ..., 0> and G = { <?, ?, ..., ?> }.

Step 3: For a positive example: delete from G every hypothesis that does not cover it, and minimally generalise S (0 becomes the value, a differing value becomes "?").

Step 4: For a negative example: replace each hypothesis in G that covers it by its minimal specialisations that still cover S, then keep only the maximally general ones.

Step 5: Display S and G after every example; stop early if G becomes empty (inconsistent data).

Step 6: Enumerate all hypotheses consistent with every example to display the complete version space.

Input (p1_enjoysport.csv)

Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Program

```
# Candidate-Elimination Algorithm - Problem 1: EnjoySport
import csv
from itertools import product



def load_csv(filename):
    """Read the CSV file: last column is the target (Yes/No)."""
    with open(filename, newline='') as f:
        reader = csv.reader(f)
        header = [h.strip() for h in next(reader)]
        rows = [[c.strip() for c in r] for r in reader if r]
    attributes = header[:-1]
    X = [r[:-1] for r in rows]
    y = [r[-1].lower() == 'yes' for r in rows]
    return attributes, X, y



def covers(h, x):
    """True if hypothesis h classifies instance x as positive."""
    return all(hi == '?' or hi == xi for hi, xi in zip(h, x))



def more_general_or_equal(h1, h2):
```

```
    """True if h1 is more general than or equal to h2."""
    return all(a == '?' or a == b or b == '0' for a, b in zip(h1, h2))



def fmt(h):
    """Print a hypothesis in textbook notation, e.g. <Sunny, ?, ?>."""
    return '<' + ', '.join(h) + '>'



def show(label, hyps):
    """Print a boundary set, one hypothesis per line."""
    if not hyps:
        print(f"   {label} = (empty)")
        return
    print(f"   {label} = " + fmt(hyps[0]))
    for h in hyps[1:]:
        print("       " + fmt(h))



def candidate_elimination(attributes, X, y):
    n = len(attributes)
    domains = [sorted(set(x[i] for x in X)) for i in range(n)]
    S = ['0'] * n                # most specific boundary
    G = [['?'] * n]              # most general boundary

    print("Initial S =", fmt(S))
    print("Initial G =", fmt(G[0]))
    print("-" * 78)

    for t, (x, positive) in enumerate(zip(X, y), start=1):
        if positive:
            # remove from G every hypothesis inconsistent with x
            G = [g for g in G if covers(g, x)]
            # minimally generalise S so that it covers x
            S = [x[i] if S[i] == '0' else (S[i] if S[i] == x[i] else '?')
                 for i in range(n)]
            label = "Positive"
        else:
            new_G = []
            for g in G:
                if not covers(g, x):
                    new_G.append(g)              # already consistent
                else:                            # minimally specialise g
                    for i in range(n):
                        if g[i] == '?':
                            for v in domains[i]:
                                if v != x[i]:
                                    h = g[:]
                                    h[i] = v
                                    if more_general_or_equal(h, S):
                                        new G.append(h)
            unique = []                          # remove duplicates
            for h in new_G:
                if h not in unique:
                    unique.append(h)
            # keep only the maximally general hypotheses
            G = [h for h in unique
                 if not any(o != h and more general or equal(o, h) for o in unique)]
            label = "Negative"

        print(f"Example {t} ({label}): {x}")
        show("S", [S])
        show("G", G)
        if not G:
            print("Version space collapsed - the data is inconsistent.")
            break

    # enumerate every hypothesis lying between S and G
    version_space = []
    for h in product(*[d + ['?'] for d in domains]):
        h = list(h)
        if all(covers(h, x) == p for x, p in zip(X, y)):
            version_space.append(h)
    return S, G, version_space
```

```
attributes, X, y = load_csv('p1_enjoysport.csv')

print("Attributes:", attributes)
print("Number of training examples:", len(X))
print()

S, G, VS = candidate elimination(attributes, X, y)

print("-" * 78)
print("Final Specific Boundary S :", fmt(S))
print("Final General Boundary G  :")
for g in G:
    print("    ", fmt(g))
print(f"\nAll {len(VS)} hypotheses in the version space:")
for h in VS:
    print("    ", fmt(h))
```

Output

```
Attributes: ['Sky', 'AirTemp', 'Humidity', 'Wind', 'Water', 'Forecast']
Number of training examples: 4

Initial S = <0, 0, 0, 0, 0, 0>
Initial G = <?, ?, ?, ?, ?, ?>
------------------------------------------------------------------------------
Example 1 (Positive): ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
   S = <Sunny, Warm, Normal, Strong, Warm, Same>
   G = <?, ?, ?, ?, ?, ?>
Example 2 (Positive): ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same']
   S = <Sunny, Warm, ?, Strong, Warm, Same>
   G = <?, ?, ?, ?, ?, ?>
Example 3 (Negative): ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change']
   S = <Sunny, Warm, ?, Strong, Warm, Same>
   G = <Sunny, ?, ?, ?, ?, ?>
       <?, Warm, ?, ?, ?, ?>
       <?, ?, ?, ?, ?, Same>
Example 4 (Positive): ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change']
   S = <Sunny, Warm, ?, Strong, ?, ?>
   G = <Sunny, ?, ?, ?, ?, ?>
       <?, Warm, ?, ?, ?, ?>
------------------------------------------------------------------------------
Final Specific Boundary S : <Sunny, Warm, ?, Strong, ?, ?>
Final General Boundary G  :
     <Sunny, ?, ?, ?, ?, ?>
     <?, Warm, ?, ?, ?, ?>

All 6 hypotheses in the version space:
     <Sunny, Warm, ?, Strong, ?, ?>
     <Sunny, Warm, ?, ?, ?, ?>
     <Sunny, ?, ?, Strong, ?, ?>
     <Sunny, ?, ?, ?, ?, ?>
     <?, Warm, ?, Strong, ?, ?>
     <?, Warm, ?, ?, ?, ?>
```

Result: S = <Sunny, Warm, ?, Strong, ?, ?>; G = <Sunny, ?, ?, ?, ?, ?>, <?, Warm, ?, ?, ?, ?>; the version space contains 6 hypothesises.

Problem 2: Customer Buys a Computer

Aim

To use the Candidate-Elimination algorithm to learn which customer profiles lead to buying a computer, using data read from a CSV file.

Problem Statement

Customer records contain Age, Income, Student status and Credit Rating, with the label BuysComputer (Yes/No). Find all hypotheses consistent with the records.

Algorithm

Step 1: Read the CSV file; store the attribute names, the examples X and the labels y (Yes = positive).

Step 2: Initialise S = <0, 0, ..., 0> and G = { <?, ?, ..., ?> }.

Step 3: For a positive example: delete from G every hypothesis that does not cover it, and minimally generalise S (0 becomes the value, a differing value becomes "?").

Step 4: For a negative example: replace each hypothesis in G that covers it by its minimal specialisations that still cover S, then keep only the maximally general ones.

Step 5: Display S and G after every example; stop early if G becomes empty (inconsistent data).

Step 6: Enumerate all hypotheses consistent with every example to display the complete version space.

Input (p2_buys_computer.csv)

Age	Income	Student	CreditRating	BuysComputer
Youth	High	No	Fair	No
Youth	Medium	Yes	Fair	Yes
Middle	Low	Yes	Fair	Yes
Senior	Medium	Yes	Excellent	No
Senior	Low	Yes	Fair	Yes
Middle	High	No	Excellent	No
Youth	Low	Yes	Fair	Yes

Program

```
# Candidate-Elimination Algorithm - Problem 2: Customer Buys a Computer
import csv
from itertools import product



def load_csv(filename):
    """Read the CSV file: last column is the target (Yes/No)."""
    with open(filename, newline='') as f:
        reader = csv.reader(f)
        header = [h.strip() for h in next(reader)]
        rows = [[c.strip() for c in r] for r in reader if r]
    attributes = header[:-1]
    X = [r[:-1] for r in rows]
    y = [r[-1].lower() == 'yes' for r in rows]
    return attributes, X, y



def covers(h, x):
    """True if hypothesis h classifies instance x as positive."""
```

```
    return all(hi == '?' or hi == xi for hi, xi in zip(h, x))



def more_general_or_equal(h1, h2):
    """True if h1 is more general than or equal to h2."""
    return all(a == '?' or a == b or b == '0' for a, b in zip(h1, h2))



def fmt(h):
    """Print a hypothesis in textbook notation, e.g. <Sunny, ?, ?>."""
    return '<' + ', '.join(h) + '>'



def show(label, hyps):
    """Print a boundary set, one hypothesis per line."""
    if not hyps:
        print(f"   {label} = (empty)")
        return
    print(f"   {label} = " + fmt(hyps[0]))
    for h in hyps[1:]:
        print("       " + fmt(h))



def candidate_elimination(attributes, X, y):
    n = len(attributes)
    domains = [sorted(set(x[i] for x in X)) for i in range(n)]
    S = ['0'] * n                 # most specific boundary
    G = [['?'] * n]               # most general boundary

    print("Initial S =", fmt(S))
    print("Initial G =", fmt(G[0]))
    print("-" * 78)

    for t, (x, positive) in enumerate(zip(X, y), start=1):
        if positive:
            # remove from G every hypothesis inconsistent with x
            G = [g for g in G if covers(g, x)]
            # minimally generalise S so that it covers x
            S = [x[i] if S[i] == '0' else (S[i] if S[i] == x[i] else '?')
                 for i in range(n)]
            label = "Positive"
        else:
            new_G = []
            for g in G:
                if not covers(g, x):
                    new_G.append(g)              # already consistent
                else:                            # minimally specialise g
                    for i in range(n):
                        if g[i] == '?':
                            for v in domains[i]:
                                if v != x[i]:
                                    h = g[:]
                                    h[i] = v
                                    if more_general_or_equal(h, S):
                                        new_G.append(h)
            unique = []                          # remove duplicates
            for h in new_G:
                if h not in unique:
                    unique.append(h)
            # keep only the maximally general hypotheses
            G = [h for h in unique
                 if not any(o != h and more_general_or_equal(o, h) for o in unique)]
            label = "Negative"

        print(f"Example {t} ({label}): {x}")
        show("S", [S])
        show("G", G)
        if not G:
            print("Version space collapsed - the data is inconsistent.")
            break

    # enumerate every hypothesis lying between S and G
    version_space = []
    for h in product(*[d + ['?'] for d in domains]):
```

```
        h = list(h)
        if all(covers(h, x) == p for x, p in zip(X, y)):
            version_space.append(h)
    return S, G, version_space



attributes, X, y = load_csv('p2 buys computer.csv')

print("Attributes:", attributes)
print("Number of training examples:", len(X))
print()

S, G, VS = candidate_elimination(attributes, X, y)

print("-" * 78)
print("Final Specific Boundary S :", fmt(S))
print("Final General Boundary G  :")
for g in G:
    print("    ", fmt(g))
print(f"\nAll {len(VS)} hypotheses in the version space:")
for h in VS:
    print("    ", fmt(h))
```

Output

```
Attributes: ['Age', 'Income', 'Student', 'CreditRating']
Number of training examples: 7

Initial S = <0, 0, 0, 0>
Initial G = <?, ?, ?, ?>
------------------------------------------------------------------------------
Example 1 (Negative): ['Youth', 'High', 'No', 'Fair']
   S = <0, 0, 0, 0>
   G = <Middle, ?, ?, ?>
       <Senior, ?, ?, ?>
       <?, Low, ?, ?>
       <?, Medium, ?, ?>
       <?, ?, Yes, ?>
       <?, ?, ?, Excellent>
Example 2 (Positive): ['Youth', 'Medium', 'Yes', 'Fair']
   S = <Youth, Medium, Yes, Fair>
   G = <?, Medium, ?, ?>
       <?, ?, Yes, ?>
Example 3 (Positive): ['Middle', 'Low', 'Yes', 'Fair']
   S = <?, ?, Yes, Fair>
   G = <?, ?, Yes, ?>
Example 4 (Negative): ['Senior', 'Medium', 'Yes', 'Excellent']
   S = <?, ?, Yes, Fair>
   G = <?, ?, Yes, Fair>
Example 5 (Positive): ['Senior', 'Low', 'Yes', 'Fair']
   S = <?, ?, Yes, Fair>
   G = <?, ?, Yes, Fair>
Example 6 (Negative): ['Middle', 'High', 'No', 'Excellent']
   S = <?, ?, Yes, Fair>
   G = <?, ?, Yes, Fair>
Example 7 (Positive): ['Youth', 'Low', 'Yes', 'Fair']
   S = <?, ?, Yes, Fair>
   G = <?, ?, Yes, Fair>
------------------------------------------------------------------------------
Final Specific Boundary S : <?, ?, Yes, Fair>
Final General Boundary G  :
     <?, ?, Yes, Fair>

All 1 hypotheses in the version space:
     <?, ?, Yes, Fair>
```

Result: S = <?, ?, Yes, Fair>; G = <?, ?, Yes, Fair>; the version space contains 1 hypothesis.

Problem 3: Edible Mushroom Classification

Aim

To apply the Candidate-Elimination algorithm to a CSV file of mushroom observations and find every hypothesis that correctly separates edible from poisonous mushrooms.

Problem Statement

Mushrooms are described by CapShape, Odor, GillSize and Habitat, and labelled Edible (Yes/No). Find the version space for the concept "edible mushroom".

Algorithm

Step 1: Read the CSV file; store the attribute names, the examples X and the labels y (Yes = positive).

Step 2: Initialise S = <0, 0, ..., 0> and G = { <?, ?, ..., ?> }.

Step 3: For a positive example: delete from G every hypothesis that does not cover it, and minimally generalise S (0 becomes the value, a differing value becomes "?").

Step 4: For a negative example: replace each hypothesis in G that covers it by its minimal specialisations that still cover S, then keep only the maximally general ones.

Step 5: Display S and G after every example; stop early if G becomes empty (inconsistent data).

Step 6: Enumerate all hypotheses consistent with every example to display the complete version space.

Input (p3_mushroom.csv)

CapShape	Odor	GillSize	Habitat	Edible
Convex	None	Broad	Woods	Yes
Flat	Foul	Narrow	Grass	No
Convex	None	Broad	Grass	Yes
Bell	None	Narrow	Woods	No
Flat	None	Broad	Woods	Yes
Bell	Pungent	Broad	Grass	No

Program

```
# Candidate-Elimination Algorithm - Problem 3: Edible Mushroom Classification
import csv
from itertools import product



def load_csv(filename):
    """Read the CSV file: last column is the target (Yes/No)."""
    with open(filename, newline='') as f:
        reader = csv.reader(f)
        header = [h.strip() for h in next(reader)]
        rows = [[c.strip() for c in r] for r in reader if r]
    attributes = header[:-1]
    X = [r[:-1] for r in rows]
    y = [r[-1].lower() == 'yes' for r in rows]
    return attributes, X, y



def covers(h, x):
    """True if hypothesis h classifies instance x as positive."""
    return all(hi == '?' or hi == xi for hi, xi in zip(h, x))
```

```
def more_general_or_equal(h1, h2):
    """True if h1 is more general than or equal to h2."""
    return all(a == '?' or a == b or b == '0' for a, b in zip(h1, h2))



def fmt(h):
    """Print a hypothesis in textbook notation, e.g. <Sunny, ?, ?>."""
    return '<' + ', '.join(h) + '>'



def show(label, hyps):
    """Print a boundary set, one hypothesis per line."""
    if not hyps:
        print(f"   {label} = (empty)")
        return
    print(f"   {label} = " + fmt(hyps[0]))
    for h in hyps[1:]:
        print("       " + fmt(h))



def candidate_elimination(attributes, X, y):
    n = len(attributes)
    domains = [sorted(set(x[i] for x in X)) for i in range(n)]
    S = ['0'] * n                # most specific boundary
    G = [['?'] * n]              # most general boundary

    print("Initial S =", fmt(S))
    print("Initial G =", fmt(G[0]))
    print("-" * 78)

    for t, (x, positive) in enumerate(zip(X, y), start=1):
        if positive:
            # remove from G every hypothesis inconsistent with x
            G = [g for g in G if covers(g, x)]
            # minimally generalise S so that it covers x
            S = [x[i] if S[i] == '0' else (S[i] if S[i] == x[i] else '?')
                 for i in range(n)]
            label = "Positive"
        else:
            new_G = []
            for g in G:
                if not covers(g, x):
                    new_G.append(g)              # already consistent
                else:                            # minimally specialise g
                    for i in range(n):
                        if g[i] == '?':
                            for v in domains[i]:
                                if v != x[i]:
                                    h = g[:]
                                    h[i] = v
                                    if more_general or equal(h, S):
                                        new G.append(h)
            unique = []                          # remove duplicates
            for h in new_G:
                if h not in unique:
                    unique.append(h)
            # keep only the maximally general hypotheses
            G = [h for h in unique
                 if not any(o != h and more_general_or_equal(o, h) for o in unique)]
            label = "Negative"

        print(f"Example {t} ({label}): {x}")
        show("S", [S])
        show("G", G)
        if not G:
            print("Version space collapsed - the data is inconsistent.")
            break

    # enumerate every hypothesis lying between S and G
    version_space = []
    for h in product(*[d + ['?'] for d in domains]):
        h = list(h)
        if all(covers(h, x) == p for x, p in zip(X, y)):
            version_space.append(h)
```

```
    return S, G, version_space



attributes, X, y = load_csv('p3_mushroom.csv')

print("Attributes:", attributes)
print("Number of training examples:", len(X))
print()

S, G, VS = candidate elimination(attributes, X, y)

print("-" * 78)
print("Final Specific Boundary S :", fmt(S))
print("Final General Boundary G  :")
for g in G:
    print("    ", fmt(g))
print(f"\nAll {len(VS)} hypotheses in the version space:")
for h in VS:
    print("    ", fmt(h))
```

Output

```
Attributes: ['CapShape', 'Odor', 'GillSize', 'Habitat']
Number of training examples: 6

Initial S = <0, 0, 0, 0>
Initial G = <?, ?, ?, ?>
------------------------------------------------------------------------------
Example 1 (Positive): ['Convex', 'None', 'Broad', 'Woods']
   S = <Convex, None, Broad, Woods>
   G = <?, ?, ?, ?>
Example 2 (Negative): ['Flat', 'Foul', 'Narrow', 'Grass']
   S = <Convex, None, Broad, Woods>
   G = <Convex, ?, ?, ?>
       <?, None, ?, ?>
       <?, ?, Broad, ?>
       <?, ?, ?, Woods>
Example 3 (Positive): ['Convex', 'None', 'Broad', 'Grass']
   S = <Convex, None, Broad, ?>
   G = <Convex, ?, ?, ?>
       <?, None, ?, ?>
       <?, ?, Broad, ?>
Example 4 (Negative): ['Bell', 'None', 'Narrow', 'Woods']
   S = <Convex, None, Broad, ?>
   G = <Convex, ?, ?, ?>
       <?, ?, Broad, ?>
Example 5 (Positive): ['Flat', 'None', 'Broad', 'Woods']
   S = <?, None, Broad, ?>
   G = <?, ?, Broad, ?>
Example 6 (Negative): ['Bell', 'Pungent', 'Broad', 'Grass']
   S = <?, None, Broad, ?>
   G = <?, None, Broad, ?>
------------------------------------------------------------------------------
Final Specific Boundary S : <?, None, Broad, ?>
Final General Boundary G  :
     <?, None, Broad, ?>

All 1 hypotheses in the version space:
     <?, None, Broad, ?>
```

Result: S = <?, None, Broad, ?>; G = <?, None, Broad, ?>; the version space contains 1 hypothesis.

Problem 4: Flu Diagnosis

Aim

To implement Candidate-Elimination on a CSV file of patient symptoms and obtain all symptom combinations consistent with a flu diagnosis.

Problem Statement

Patient records contain Fever, Cough, Headache, BodyPain and Fatigue, with the diagnosis Flu (Yes/No). Find the S boundary, G boundary and version space.

Algorithm

Step 1: Read the CSV file; store the attribute names, the examples X and the labels y (Yes = positive).

Step 2: Initialise S = <0, 0, ..., 0> and G = { <?, ?, ..., ?> }.

Step 3: For a positive example: delete from G every hypothesis that does not cover it, and minimally generalise S (0 becomes the value, a differing value becomes "?").

Step 4: For a negative example: replace each hypothesis in G that covers it by its minimal specialisations that still cover S, then keep only the maximally general ones.

Step 5: Display S and G after every example; stop early if G becomes empty (inconsistent data).

Step 6: Enumerate all hypotheses consistent with every example to display the complete version space.

Input (p4_flu.csv)

Fever	Cough	Headache	BodyPain	Fatigue	Flu
High	Yes	Yes	Yes	Yes	Yes
Normal	No	No	No	No	No
High	Yes	No	Yes	No	Yes
High	No	Yes	Yes	Yes	No
Normal	Yes	Yes	No	Yes	No
High	Yes	Yes	No	No	Yes

Program

```
# Candidate-Elimination Algorithm - Problem 4: Flu Diagnosis
import csv
from itertools import product



def load_csv(filename):
    """Read the CSV file: last column is the target (Yes/No)."""
    with open(filename, newline='') as f:
        reader = csv.reader(f)
        header = [h.strip() for h in next(reader)]
        rows = [[c.strip() for c in r] for r in reader if r]
    attributes = header[:-1]
    X = [r[:-1] for r in rows]
    y = [r[-1].lower() == 'yes' for r in rows]
    return attributes, X, y



def covers(h, x):
    """True if hypothesis h classifies instance x as positive."""
    return all(hi == '?' or hi == xi for hi, xi in zip(h, x))
```

```
def more_general_or_equal(h1, h2):
    """True if h1 is more general than or equal to h2."""
    return all(a == '?' or a == b or b == '0' for a, b in zip(h1, h2))



def fmt(h):
    """Print a hypothesis in textbook notation, e.g. <Sunny, ?, ?>."""
    return '<' + ', '.join(h) + '>'



def show(label, hyps):
    """Print a boundary set, one hypothesis per line."""
    if not hyps:
        print(f"   {label} = (empty)")
        return
    print(f"   {label} = " + fmt(hyps[0]))
    for h in hyps[1:]:
        print("       " + fmt(h))



def candidate_elimination(attributes, X, y):
    n = len(attributes)
    domains = [sorted(set(x[i] for x in X)) for i in range(n)]
    S = ['0'] * n                # most specific boundary
    G = [['?'] * n]              # most general boundary

    print("Initial S =", fmt(S))
    print("Initial G =", fmt(G[0]))
    print("-" * 78)

    for t, (x, positive) in enumerate(zip(X, y), start=1):
        if positive:
            # remove from G every hypothesis inconsistent with x
            G = [g for g in G if covers(g, x)]
            # minimally generalise S so that it covers x
            S = [x[i] if S[i] == '0' else (S[i] if S[i] == x[i] else '?')
                 for i in range(n)]
            label = "Positive"
        else:
            new_G = []
            for g in G:
                if not covers(g, x):
                    new_G.append(g)              # already consistent
                else:                            # minimally specialise g
                    for i in range(n):
                        if g[i] == '?':
                            for v in domains[i]:
                                if v != x[i]:
                                    h = g[:]
                                    h[i] = v
                                    if more_general or equal(h, S):
                                        new G.append(h)
            unique = []                          # remove duplicates
            for h in new_G:
                if h not in unique:
                    unique.append(h)
            # keep only the maximally general hypotheses
            G = [h for h in unique
                 if not any(o != h and more_general_or_equal(o, h) for o in unique)]
            label = "Negative"

        print(f"Example {t} ({label}): {x}")
        show("S", [S])
        show("G", G)
        if not G:
            print("Version space collapsed - the data is inconsistent.")
            break

    # enumerate every hypothesis lying between S and G
    version_space = []
    for h in product(*[d + ['?'] for d in domains]):
        h = list(h)
        if all(covers(h, x) == p for x, p in zip(X, y)):
            version_space.append(h)
```

```
    return S, G, version_space



attributes, X, y = load_csv('p4_flu.csv')

print("Attributes:", attributes)
print("Number of training examples:", len(X))
print()

S, G, VS = candidate elimination(attributes, X, y)

print("-" * 78)
print("Final Specific Boundary S :", fmt(S))
print("Final General Boundary G  :")
for g in G:
    print("    ", fmt(g))
print(f"\nAll {len(VS)} hypotheses in the version space:")
for h in VS:
    print("    ", fmt(h))
```

Output

```
Attributes: ['Fever', 'Cough', 'Headache', 'BodyPain', 'Fatigue']
Number of training examples: 6

Initial S = <0, 0, 0, 0, 0>
Initial G = <?, ?, ?, ?, ?>
------------------------------------------------------------------------------
Example 1 (Positive): ['High', 'Yes', 'Yes', 'Yes', 'Yes']
   S = <High, Yes, Yes, Yes, Yes>
   G = <?, ?, ?, ?, ?>
Example 2 (Negative): ['Normal', 'No', 'No', 'No', 'No']
   S = <High, Yes, Yes, Yes, Yes>
   G = <High, ?, ?, ?, ?>
       <?, Yes, ?, ?, ?>
       <?, ?, Yes, ?, ?>
       <?, ?, ?, Yes, ?>
       <?, ?, ?, ?, Yes>
Example 3 (Positive): ['High', 'Yes', 'No', 'Yes', 'No']
   S = <High, Yes, ?, Yes, ?>
   G = <High, ?, ?, ?, ?>
       <?, Yes, ?, ?, ?>
       <?, ?, ?, Yes, ?>
Example 4 (Negative): ['High', 'No', 'Yes', 'Yes', 'Yes']
   S = <High, Yes, ?, Yes, ?>
   G = <?, Yes, ?, ?, ?>
Example 5 (Negative): ['Normal', 'Yes', 'Yes', 'No', 'Yes']
   S = <High, Yes, ?, Yes, ?>
   G = <High, Yes, ?, ?, ?>
       <?, Yes, ?, Yes, ?>
Example 6 (Positive): ['High', 'Yes', 'Yes', 'No', 'No']
   S = <High, Yes, ?, ?, ?>
   G = <High, Yes, ?, ?, ?>
------------------------------------------------------------------------------
Final Specific Boundary S : <High, Yes, ?, ?, ?>
Final General Boundary G  :
     <High, Yes, ?, ?, ?>

All 1 hypotheses in the version space:
     <High, Yes, ?, ?, ?>
```

Result: S = <High, Yes, ?, ?, ?>; G = <High, Yes, ?, ?, ?>; the version space contains 1 hypothesis.

Problem 5: Student Placement Prediction

Aim

To use the Candidate-Elimination algorithm on a CSV file of student records to find all hypotheses that explain which students get placed.

Problem Statement

Student records contain CGPA, Internship, Communication and Projects, with the result Placed (Yes/No). Find the S boundary, the G boundary and every hypothesis in between.

Algorithm

Step 1: Read the CSV file; store the attribute names, the examples X and the labels y (Yes = positive).

Step 2: Initialise S = <0, 0, ..., 0> and G = { <?, ?, ..., ?> }.

Step 3: For a positive example: delete from G every hypothesis that does not cover it, and minimally generalise S (0 becomes the value, a differing value becomes "?").

Step 4: For a negative example: replace each hypothesis in G that covers it by its minimal specialisations that still cover S, then keep only the maximally general ones.

Step 5: Display S and G after every example; stop early if G becomes empty (inconsistent data).

Step 6: Enumerate all hypotheses consistent with every example to display the complete version space.

Input (p5_placement.csv)

CGPA	Internship	Communication	Projects	Placed
High	Yes	Good	Many	Yes
Low	No	Poor	Few	No
High	Yes	Good	Few	Yes
Medium	Yes	Poor	Many	No
High	No	Good	Many	No
High	Yes	Good	Many	Yes

Program

```
# Candidate-Elimination Algorithm - Problem 5: Student Placement
import csv
from itertools import product



def load_csv(filename):
    """Read the CSV file: last column is the target (Yes/No)."""
    with open(filename, newline='') as f:
        reader = csv.reader(f)
        header = [h.strip() for h in next(reader)]
        rows = [[c.strip() for c in r] for r in reader if r]
    attributes = header[:-1]
    X = [r[:-1] for r in rows]
    y = [r[-1].lower() == 'yes' for r in rows]
    return attributes, X, y



def covers(h, x):
    """True if hypothesis h classifies instance x as positive."""
    return all(hi == '?' or hi == xi for hi, xi in zip(h, x))
```

```
def more_general_or_equal(h1, h2):
    """True if h1 is more general than or equal to h2."""
    return all(a == '?' or a == b or b == '0' for a, b in zip(h1, h2))



def fmt(h):
    """Print a hypothesis in textbook notation, e.g. <Sunny, ?, ?>."""
    return '<' + ', '.join(h) + '>'



def show(label, hyps):
    """Print a boundary set, one hypothesis per line."""
    if not hyps:
        print(f"   {label} = (empty)")
        return
    print(f"   {label} = " + fmt(hyps[0]))
    for h in hyps[1:]:
        print("       " + fmt(h))



def candidate_elimination(attributes, X, y):
    n = len(attributes)
    domains = [sorted(set(x[i] for x in X)) for i in range(n)]
    S = ['0'] * n                # most specific boundary
    G = [['?'] * n]              # most general boundary

    print("Initial S =", fmt(S))
    print("Initial G =", fmt(G[0]))
    print("-" * 78)

    for t, (x, positive) in enumerate(zip(X, y), start=1):
        if positive:
            # remove from G every hypothesis inconsistent with x
            G = [g for g in G if covers(g, x)]
            # minimally generalise S so that it covers x
            S = [x[i] if S[i] == '0' else (S[i] if S[i] == x[i] else '?')
                 for i in range(n)]
            label = "Positive"
        else:
            new_G = []
            for g in G:
                if not covers(g, x):
                    new_G.append(g)              # already consistent
                else:                            # minimally specialise g
                    for i in range(n):
                        if g[i] == '?':
                            for v in domains[i]:
                                if v != x[i]:
                                    h = g[:]
                                    h[i] = v
                                    if more_general_or_equal(h, S):
                                        new_G.append(h)
            unique = []                          # remove duplicates
            for h in new_G:
                if h not in unique:
                    unique.append(h)
            # keep only the maximally general hypotheses
            G = [h for h in unique
                 if not any(o != h and more_general_or_equal(o, h) for o in unique)]
            label = "Negative"

        print(f"Example {t} ({label}): {x}")
        show("S", [S])
        show("G", G)
        if not G:
            print("Version space collapsed - the data is inconsistent.")
            break

    # enumerate every hypothesis lying between S and G
    version_space = []
    for h in product(*[d + ['?'] for d in domains]):
        h = list(h)
        if all(covers(h, x) == p for x, p in zip(X, y)):
            version_space.append(h)
```

```
    return S, G, version_space



attributes, X, y = load_csv('p5_placement.csv')

print("Attributes:", attributes)
print("Number of training examples:", len(X))
print()

S, G, VS = candidate elimination(attributes, X, y)

print("-" * 78)
print("Final Specific Boundary S :", fmt(S))
print("Final General Boundary G  :")
for g in G:
    print("    ", fmt(g))
print(f"\nAll {len(VS)} hypotheses in the version space:")
for h in VS:
    print("    ", fmt(h))
```

Output

```
Attributes: ['CGPA', 'Internship', 'Communication', 'Projects']
Number of training examples: 6

Initial S = <0, 0, 0, 0>
Initial G = <?, ?, ?, ?>
------------------------------------------------------------------------------
Example 1 (Positive): ['High', 'Yes', 'Good', 'Many']
   S = <High, Yes, Good, Many>
   G = <?, ?, ?, ?>
Example 2 (Negative): ['Low', 'No', 'Poor', 'Few']
   S = <High, Yes, Good, Many>
   G = <High, ?, ?, ?>
       <?, Yes, ?, ?>
       <?, ?, Good, ?>
       <?, ?, ?, Many>
Example 3 (Positive): ['High', 'Yes', 'Good', 'Few']
   S = <High, Yes, Good, ?>
   G = <High, ?, ?, ?>
       <?, Yes, ?, ?>
       <?, ?, Good, ?>
Example 4 (Negative): ['Medium', 'Yes', 'Poor', 'Many']
   S = <High, Yes, Good, ?>
   G = <High, ?, ?, ?>
       <?, ?, Good, ?>
Example 5 (Negative): ['High', 'No', 'Good', 'Many']
   S = <High, Yes, Good, ?>
   G = <High, Yes, ?, ?>
       <?, Yes, Good, ?>
Example 6 (Positive): ['High', 'Yes', 'Good', 'Many']
   S = <High, Yes, Good, ?>
   G = <High, Yes, ?, ?>
       <?, Yes, Good, ?>
------------------------------------------------------------------------------
Final Specific Boundary S : <High, Yes, Good, ?>
Final General Boundary G  :
     <High, Yes, ?, ?>
     <?, Yes, Good, ?>

All 3 hypotheses in the version space:
     <High, Yes, Good, ?>
     <High, Yes, ?, ?>
     <?, Yes, Good, ?>
```

Result: S = <High, Yes, Good, ?>; G = <High, Yes, ?, ?>, <?, Yes, Good, ?>; the version space contains 3 hypothesises.